💻 Ejemplos de desarrollo código, configuraciones y scripts

Codificar con agentes · Parte 4 · Octubre 2026

👤 Sergi Garcia Barea

CC BY-SA 4.0

🖥️ Ejemplo 1: componente frontend con validación

📝 PROMPT

```
"Crea el componente Registro
con campos:

- Nombre, email, DNI y contraseña
- Validación en vivo: email con
  regex, DNI válido (letra correcta),
  contraseña de al menos 8 caracteres
- Mensaje de error bajo cada campo,
  accesible con aria-describedby
- Estado reutilizando el composable
  que ya usamos en el proyecto

Añade pocas pruebas unitarias de
los validadores."
```

OpenCode · Frontend · Accesible

💡 QUÉ ENSEÑA ESTE EJEMPLO

- ✅ Enumera campos y reglas exactas → nada que adivinar
- ✅ Accesibilidad pedida de fábrica, no como parche
- ✅ "los composables que ya usamos" → se apoya en el código existente
- ✅ Tests incluidos en la misma tarea
- ✅ Siguiente paso natural: "revisa el diff y commita"

🔊

🌐 Ejemplo 2: API REST con CRUD completo

📝 PROMPT

```
"Añade el recurso /habitos con CRUD:

- Modelo: id, título, categoría,
  frecuencia (diaria/semanal), fecha
  de inicio
- Endpoints: GET lista con paginación
  y filtro por categoría, GET por id,
  POST, PUT y DELETE
- Validación: título obligatorio,
  frecuencia limitada a dos valores
- Errores en format consistente
  { error: { message, detail } }
- Códigos HTTP correctos (201, 204, 400, 404)

Sigue el estilo del módulo
de usuarios. Tests de integración
incluidos."
```

OpenCode · Backend · REST

💡 DETALLES QUE MARCAN LA DIFERENCIA

- ✅ Formato del error exacto → frontend y agente comparten contrato
- ✅ Códigos HTTP a mano → disciplina REST de verdad
- ✅ "Sigue el estilo del módulo de usuarios" → consistencia automática
- ✅ Paginación pedida de inicio, no en el paso 2
- ✅ Tests de integración: el agente levanta lo que hace falta

🔊

🗄️ Ejemplo 3: base de datos con el agente

📝 PROMPT

```
"Genera la migración para el módulo
de 'asistencias':

- Tabla asistencia (id, clase_id,
  alumno_id, fecha, estado)
- estado: enum presente / tarde /
  ausente / justificada
- FK a clase y alumno con ON DELETE
  RESTRICT
- Índice único (clase_id, alumno_id, fecha)
- Datos de prueba: 3 clases, 25 alumnos,
  1 mes de asistencias realistas

Déjala aplicada en local con docker compose."
```

OpenCode · BD · Migración + seed

💡 DÓNDE ACIERTA EL PROMPT

- ✅ El modelo sale explícito, con tipos y reglas
- ✅ Integridad referencial: qué pasa al borrar (RESTRICT)
- ✅ Índice único evita duplicar la asistencia del mismo día
- ✅ Datos de prueba realistas → las demos son creíbles
- ✅ "aplicada en local" → el agente arranca el compose y ejecuta

🔊

🧪 Ejemplo 4: tests para código que ya existe

📝 PROMPT

```
"El módulo src/fechas/ calcula
días hábiles. Tiene 0 tests.

1. Léelo y dime sus casos límite
2. Escribe tests con pytest para:
- festivos nacionales
   - rangos que cruzan el fin de semana
   - rangos al revés (inicio > fin)
   - 29 de febrero
3. Si encuentras un bug en el módulo,
   NO lo arregles: anótalo en un
   reporte aparte y dime qué test
   lo delata"
```

OpenCode · Tests · Caracterización

💡 EL PATRÓN "CHARACTERIZATION"

- ✅ Antes de tocar código viejo: red de tests que lo envuelva
- ✅ El agente descubre casos límite que tú no pensaste
- ✅ "No arregles, notifica" → cero sorpresas en el paso siguiente
- ✅ Con tests verdes, el refactor es llevadero
- ✅ Trabaja igual en Java, Python o TypeScript

🔊

🧹 Ejemplo 5: refactor con red de seguridad

📝 PROMPT

```
"El fichero pedido.py tiene un
cálculo de importe de 180 líneas.

1. Extrae a funciones puras los
   cálculos: base, descuentos
   e impuestos
2. Nombres públicos y claros
3. No cambies el comportamiento:
   los tests del módulo deben seguir
   pasando, sin tocar ni una cifra
4. Al final: resumen del antes
   y del después"
```

OpenCode · Refactor · Safe

💡 REFACTOR QUE NO ROMPE

- ✅ "No cambies el comportamiento" → el refactor es mecánico y verificable
- ✅ Funciones puras primero: más fáciles de testear
- ✅ El diff queda corto y explicable
- ✅ Commit limpio: solo refactor, sin cambio de comportamiento
- ✅ Pequeño paso siguiente: "ahora añade el test del descuento acumulado"

🔊

🐛 Ejemplo 6: depurar desde un traceback

📝 PROMPT

```
"Al importar un CSV de 500 filas
me sale esto:

ValueError: could not convert
string to float: '3,5'
   in parser.py:44, en calcular_media

1. Encuentra la causa exacta
2. Propón el arreglo mínimo
3. Añade un test que reproduczca el
   caso (coma decimal)
4. Dime si hay más sitios con el
   mismo patrón"
```

OpenCode · Debug · Root cause

💡 DEPURAR CON EL AGENTE

- ✅ El traceback completo + el fichero real: sin descripciones vagas
- ✅ "Causa exacta" evita el parche que disimula el bug
- ✅ Pide el test del caso → el bug no vuelve
- ✅ "¿Más sitios con este patrón?" → barre el código de golpe
- ✅ Funciona igual con errores de compilación, de test o de runtime

🔊

📄 Ejemplo 7: documentación del proyecto

📝 PROMPT

```
"Actualiza la documentación:

- README: intro, requisitos,
  arranque en 3 pasos y captura
- docs/arquitectura.md: en qué
  anda cada capa y cómo se conecta
  (con un diagrama Mermaid)
- Docs en español, nombres de
código en inglés

Si algo NO puedes confirmarlo
leyendo el código, márcalo con
⚠️ por revisar."
```

OpenCode · Docs · Mermaid

💡 DOCUMENTACIÓN QUE NO MIENTE

- ✅ El agente la genera leyendo el código real, no la idea
- ✅ Diagrama Mermaid: se actualiza igual que el código
- ✅ "Márcalo con ⚠️" → detecta el punto que puede estar inventado
- ✅ Ideal antes de una entrega o de un cambio de equipo
- ✅ Reviso tú el resultado: la doc también se revisa

🔊

🧩 Qué tienen en común estos prompts

📍 Ancorados al repo real
"sigue el módulo de usuarios", "los tests del módulo", "docker compose" — nada genérico.

📏 Alcance en una frase
¿Qué ficheros puede tocar? ¿Qué compilados/tests deben pasar? ¿Cuándo parar?

✅ Verificables
Cada prompt incluye cómo comprobar que ha ido bien: tests, arranque, códigos HTTP.

🚦 Con reglas de seguridad
"no arregles, notifica" · "no tocar legacy" · "máximo 3 reintentos" — el agente sabe frenar.

📤 Formato de salida pedido
"resumen del antes y del después", "un test que reproduzca el caso", "marcado con ⚠️" — el resultado llega listo para leer.

🎯 Cualquier ejemplo de este deck se pega en tu proyecto cambiando dominio, stack y comandos.

🔊

⚙️ Ejemplo 8: configuración y despliegue

📝 PROMPT

```
"Prepara el proyecto para desplegar:

- Dockerfile multi-stage (build +
  imagen final con Python slim)
- .dockerignore correcto
- docker-compose con app, BD y
  healthchecks
- .env.example con TODAS las
  variables que lee la app, sin
  valores reales

Verifica con docker compose up
que la app levanta y responde
en /health."
```

OpenCode · DevOps · Configuración

💡 CONFIGURACIÓN = CÓDIGO

- ✅ Las configuraciones son texto: los LLM las generan muy bien si das el objetivo claro
- ✅ "TODAS las variables" hace que el agente localice cada env que se ha quedado fuera
- ✅ .env.example comitado, .env descartado
- ✅ Healthcheck: el compose arranca sabiendo si la app está sana
- ✅ Esto se profundiza en la parte 6 👉

🔊

✨ En esta parte 4

🖥️ Código
Componentes · CRUDs · modelos · pantallas — anclados a tu repo y estilo

🗄️ Datos
Migraciones, modelos relacionales, seeds realistas — integridad pedida de fábrica

🧪 Calidad
Tests de caracterización · refactors con red de seguridad · depuración por root cause

⚙️ Configuración
Dockerfile · compose · .env.example — la infra también es código

Siguiente: **Plugins y MCPs** — dar superpoderes al agente: GitHub, bases de datos, navegador… 🔌

CC BY-SA 4.0 · Sergi Garcia Barea · Octubre 2026

🔊